Dependable Distributed Systems
Master of Science in Engineering in Computer Science

AA 2026/2027

LECTURE 13: TOTAL ORDER BROADCAST

System model

Static set of processes $\Pi = \{p_1 \ldots p_n\}$

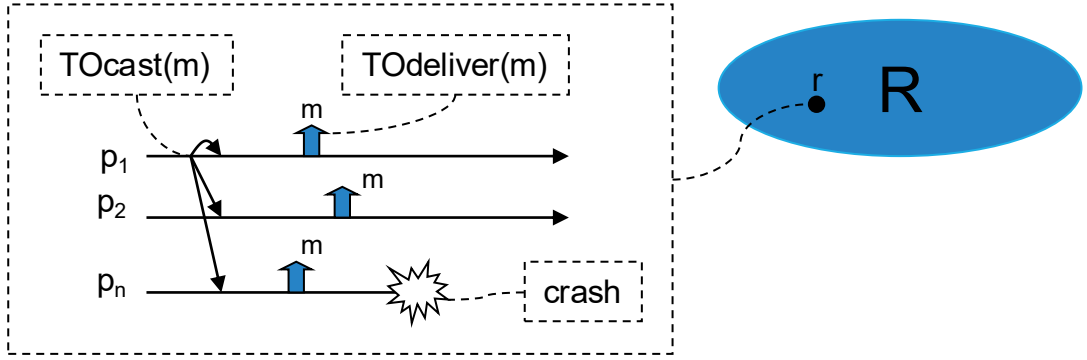
Message passing over perfect channels

- i.e., message exchanged between correct processes are reliably delivered

Asynchronous

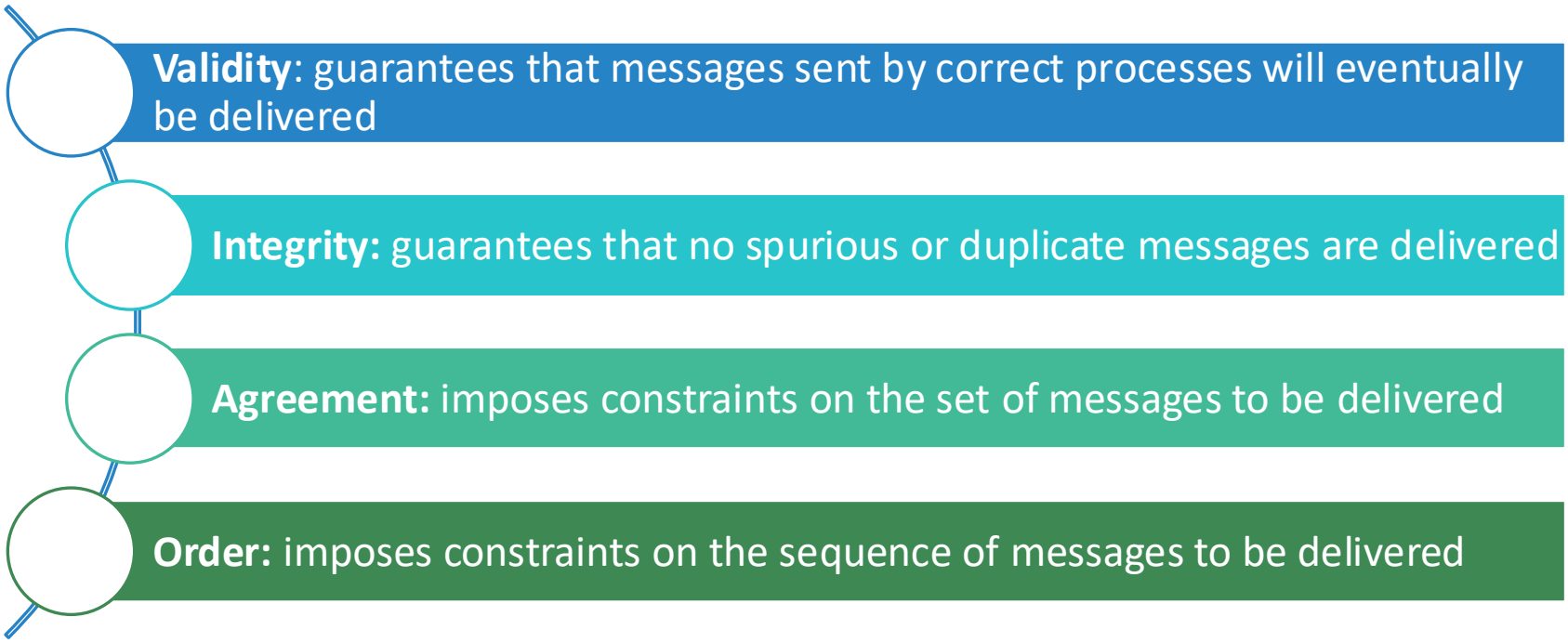
Crash fault model for processes

We characterize the system in terms of its possible runs R



TO specifications

Total order specifications are usually composed by four properties



TO specifications

Total Order Broadcast = TO(V,I,A,O)

- V = Validity
- I = Integrity
- A = Agreement
- O = Order

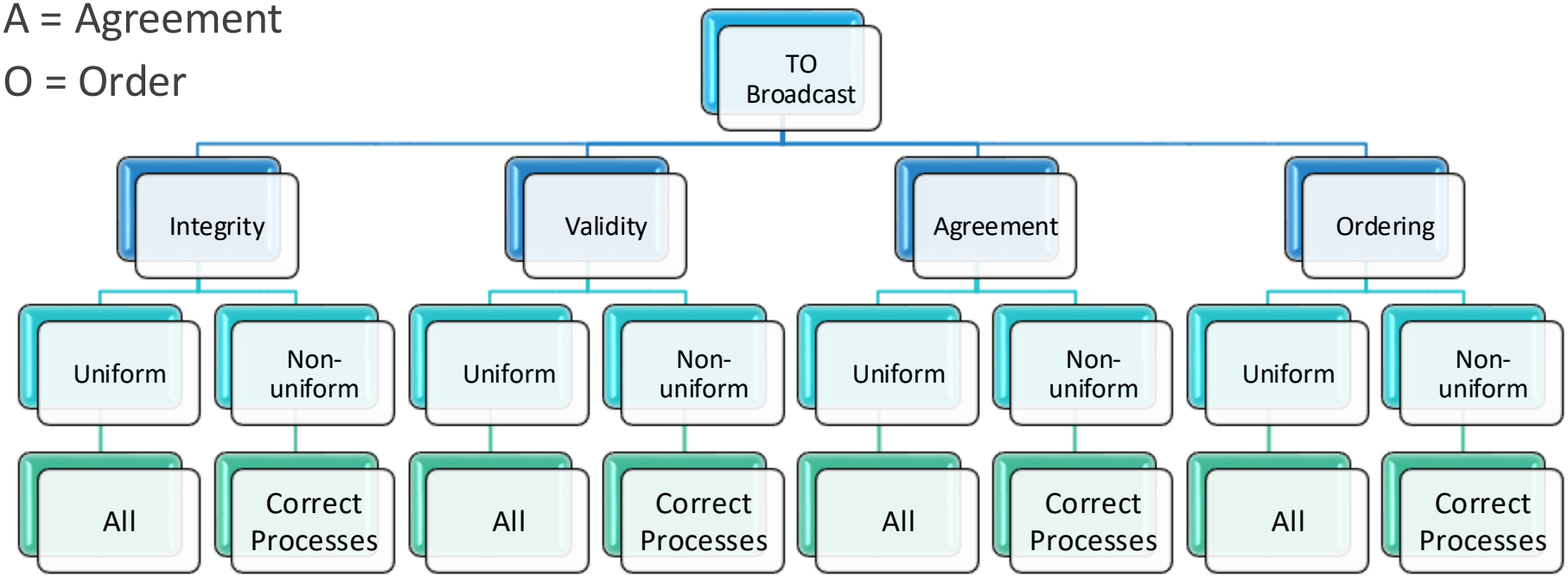
Distinct specifications arise from distinct formulations of each property

- uniform vs non-uniform
- A uniform property imposes restrictions on the behavior of (at least) correct processes

TO specifications

Total Order Broadcast = TO(V,I,A,O)

- V = Validity
- I = Integrity
- A = Agreement
- O = Order



TO Specifications

Crash failure + Perfect channels $\Rightarrow$

- **NUV:** if a correct process TOCAST a message m then some *correct* process will eventually deliver m
- **UI**: For any message m, every process p delivers m at most once and only if m was previously TOCAST by some (correct or not) process

TO specifications

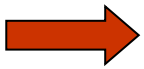
Total Order Broadcast = TO(V,I,A,O)

- V = Validity
- I = Integrity

NUV

UI

- A = Agreement
- O = Order

 TO(A,O)

Distinct specifications arise from distinct formulations of each property

- uniform vs non-uniform
- A uniform property imposes restrictions on the behavior of (at least) correct processes

The Agreement property

UNIFORM AGREEMENT (UA)

If a process (correct or not) TODelivers a message m, then all correct processes will eventually TODeliver m

NON-UNIFORM AGREEMENT (NUA)

If a correct process TODelivers a message m, then all correct processes will eventually TODeliver m

CONSTRAINS THE SET OF DELIVERED MESSAGES

Correct processes always deliver the same set of messages M

Each faulty process p delivers a set M_p

UA: $M_p \subseteq M$

NUA: M_p *can be* s.t. $M_p - M \neq \emptyset$

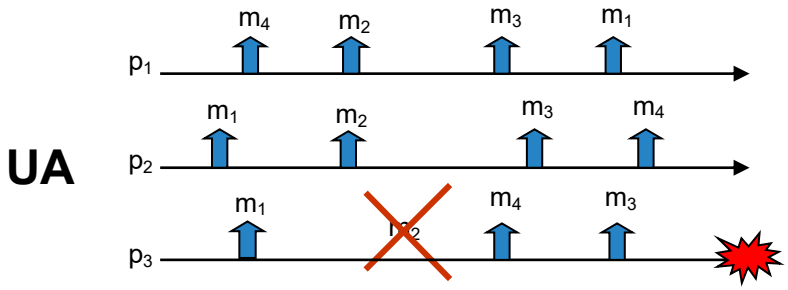
The Agreement property

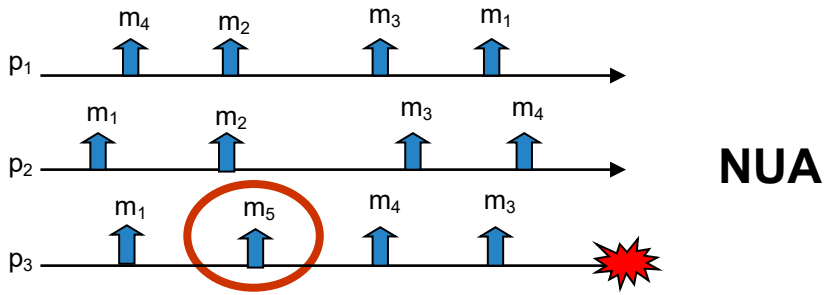
UNIFORM AGREEMENT (UA)

If a process (correct or not) TODelivers a message m, then all correct processes will eventually TODeliver m

NON-UNIFORM AGREEMENT (NUA)

If a correct process TODelivers a message m, then all correct processes will eventually TODeliver m





The Ordering Property

STRONG UNIFORM TOTAL ORDER (SUTO)

If some process TODelivers some message m before message m', then a process TODelivers m' only after it has TODelivered m.



- same order
- same prefix of the set of delivered messages
- after an omission, disjoint sets of delivered messages

WEAK UNIFORM TOTAL ORDER (WUTO)

If process p and process q both TODdeliver messages m and m', then p TODelivers m before m' if and only if q TODdelivers m before m'.



- no restrictions on the set of delivered messages

The Order Property

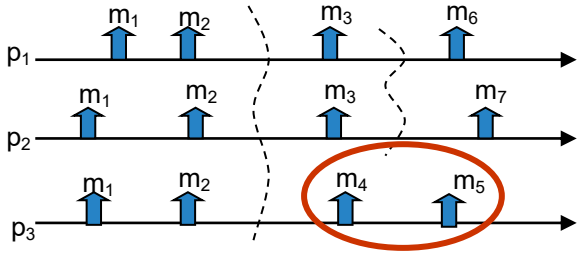
STRONG UNIFORM TOTAL ORDER (SUTO)

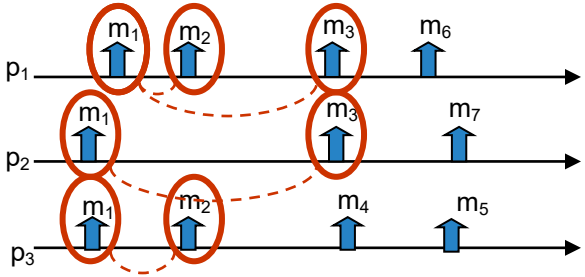
If some process TODelivers some message m before message m', then a process TODelivers m' only after it has TODelivered m.

WEAK UNIFORM TOTAL ORDER (WUTO)

If process p and process q both TODdeliver messages m and m', then p TODelivers m before m' if and only if q TODdelivers m before m'.

SUTO





WUTO

The Order Property

SUTO and WUTO are uniform but they both have a non-uniform counterpart

STRONG NON-UNIFORM TOTAL ORDER (SNUTO)

If some correct process TODelivers some message m before message m', then a correct process TODelivers m' only after it has TODelivered m.

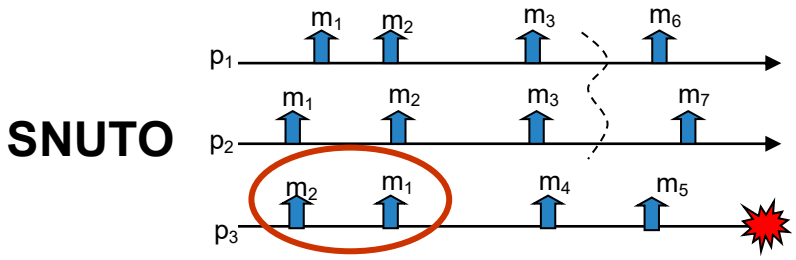
WEAK NON-UNIFORM TOTAL ORDER (WNUTO)

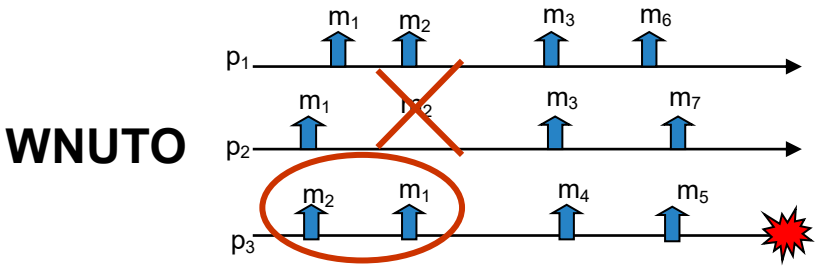
If correct processes p and q both TODeliver messages m and m', then p TODelivers m before m' if and only if q TODelivers m before m'.

The Order property (2)

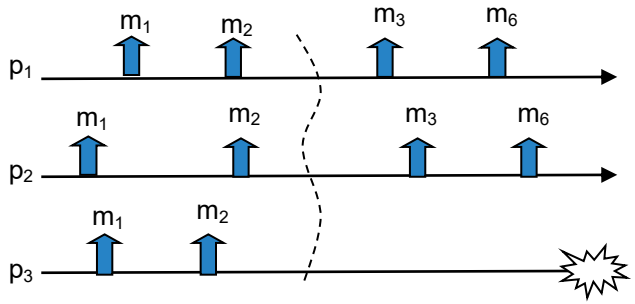
SUTO $\Rightarrow$ WUTO

SNUTO $\Rightarrow$ WNUTO





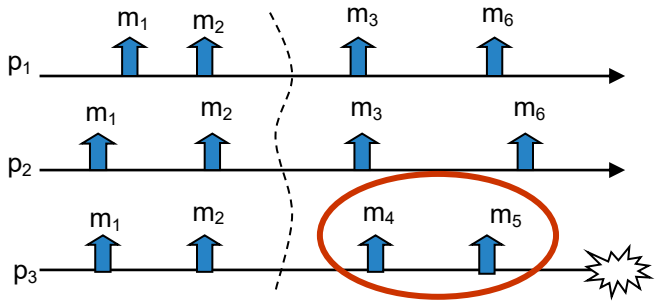
TO specifications

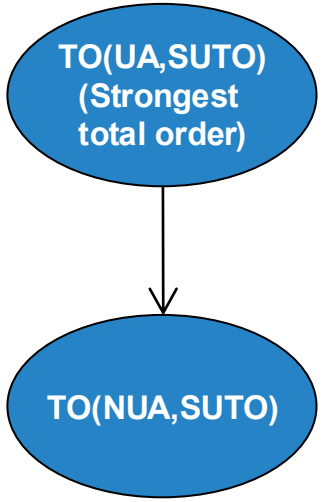


TO(UA,SUTO)

- The strongest TO spec.

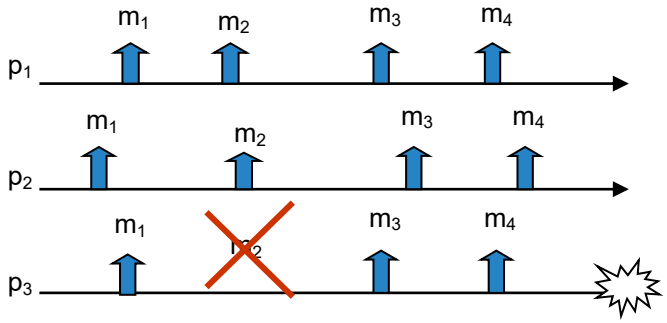
TO(NUA,SUTO)

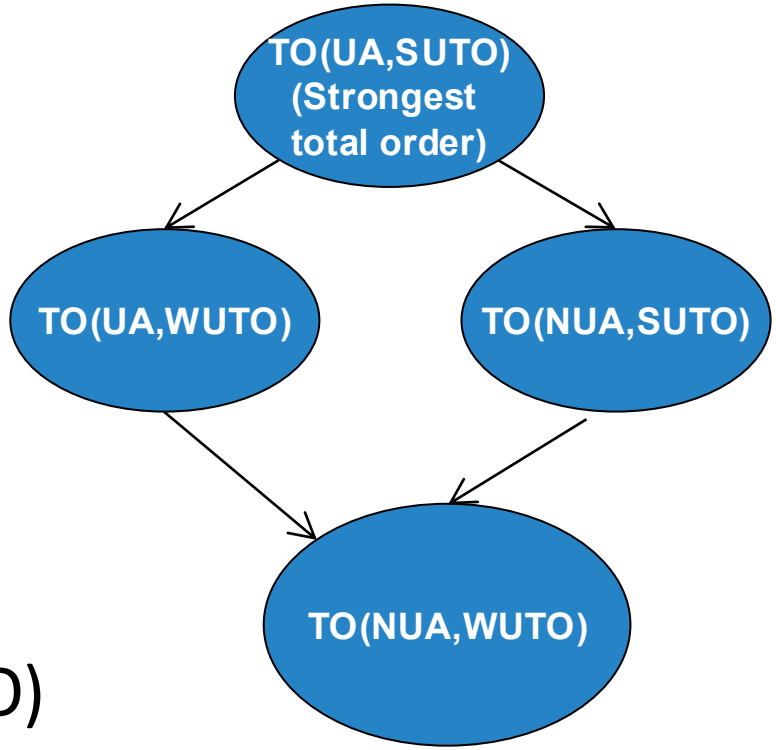


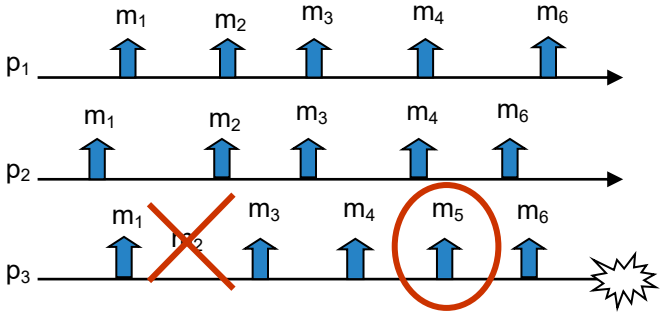


TO specifications (2)

TO(UA,WUTO)



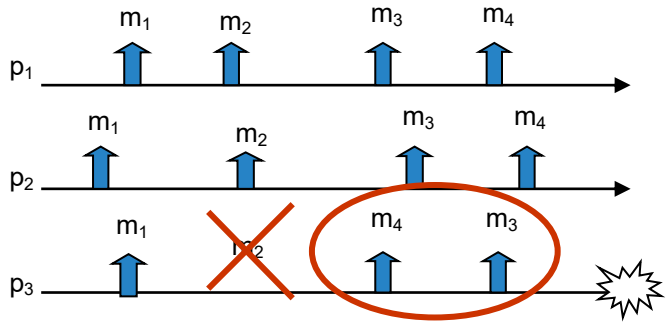




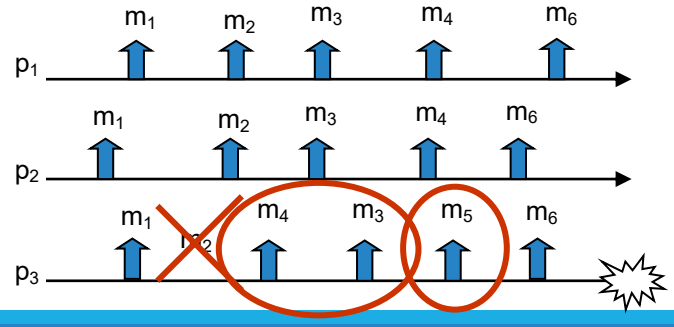
TO(NUA,WUTO)

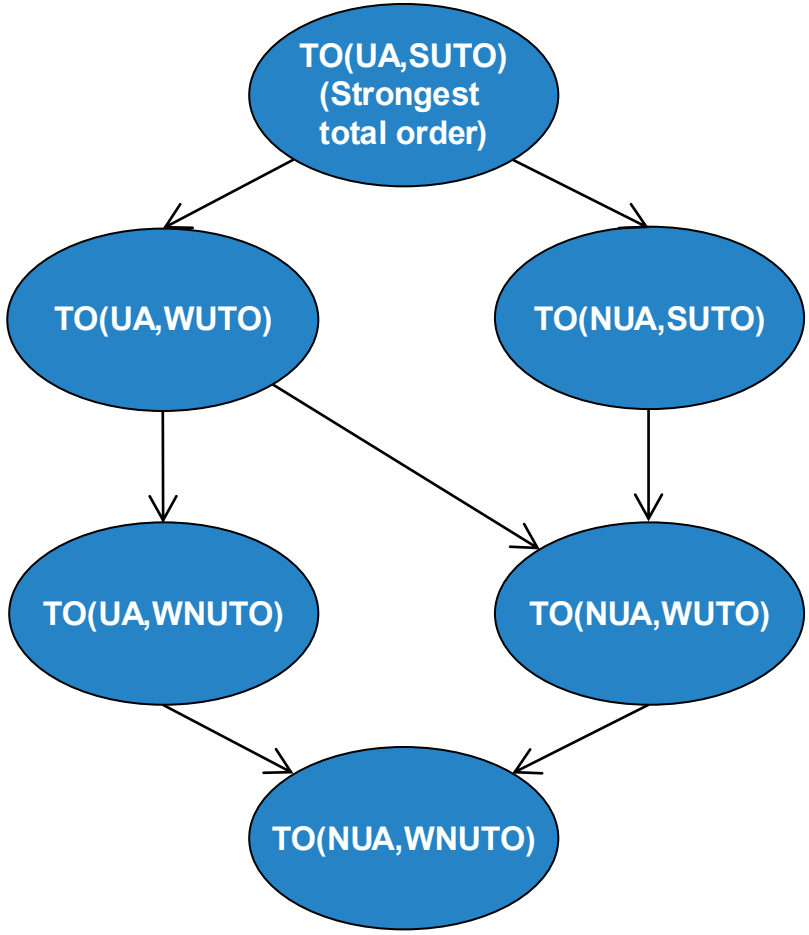
TO specifications (3)

TO(UA,WNUTO)

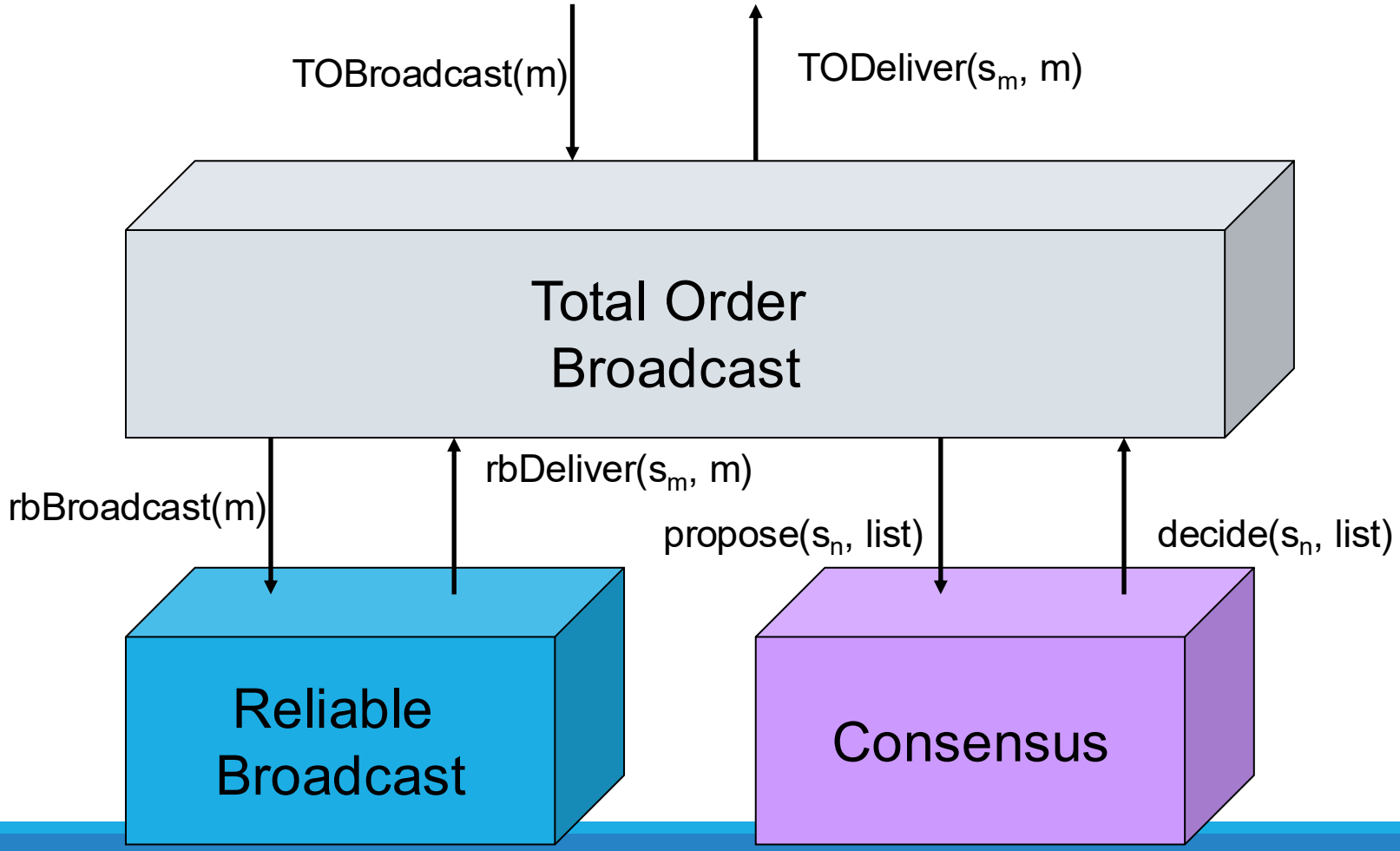


TO(NUA,WNUTO)





Total Order Implementation



Total Order Algorithm

Algorithm 6.1: Consensus-Based Total-Order Broadcast

Implements:
TotalOrderBroadcast, **instance** *tob*.

Uses:
ReliableBroadcast, **instance** *rb*;
Consensus (multiple instances).

upon event $\langle$ *tob*, *Init* $\rangle$ **do**
unordered := $\emptyset$;
delivered := $\emptyset$;
round := 1;
wait := FALSE;

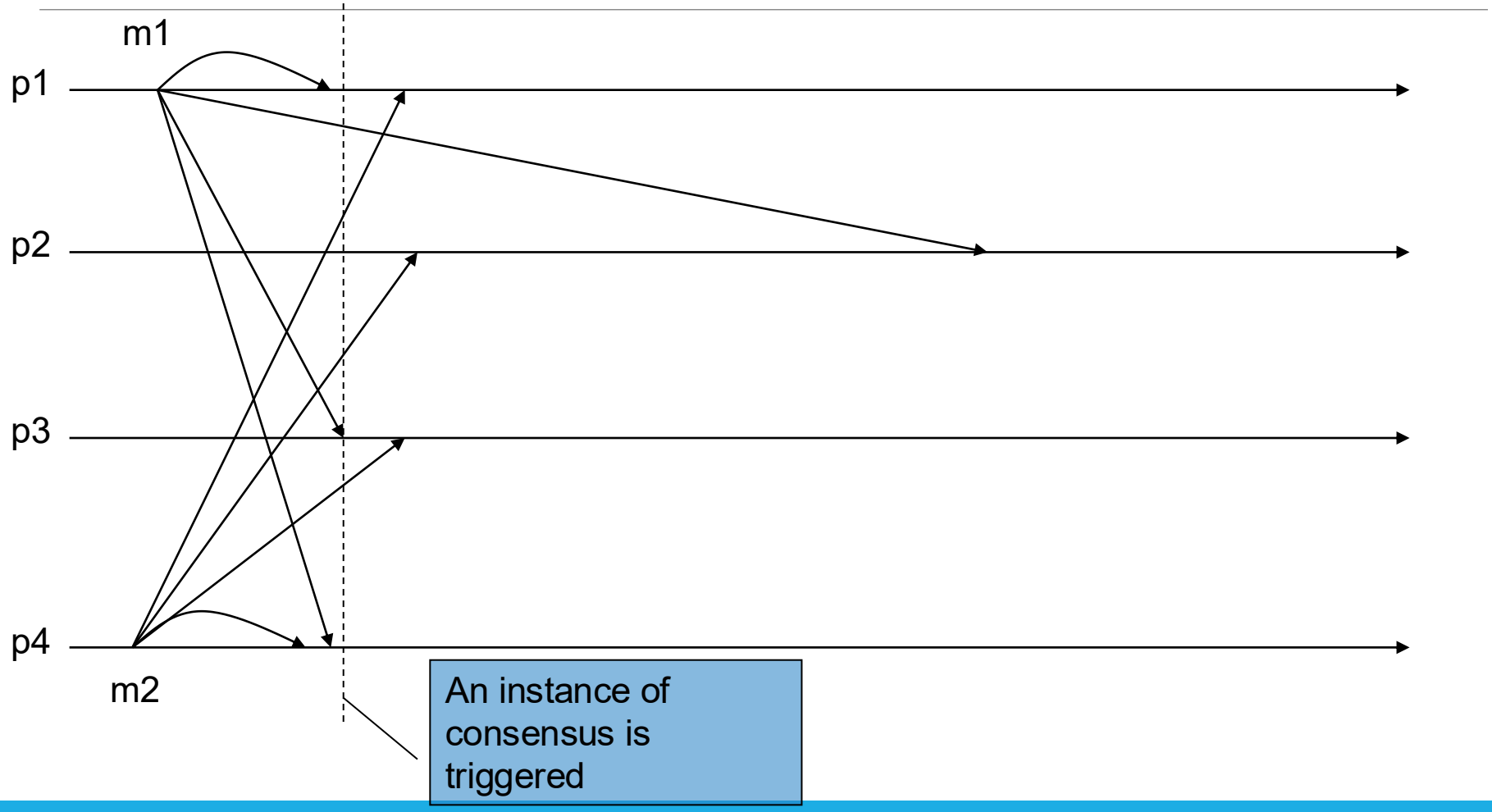
upon event $\langle$ *tob*, *Broadcast* | m $\rangle$ **do**
trigger $\langle$ *rb*, *Broadcast* | m $\rangle$;

upon event $\langle$ *rb*, *Deliver* | p, m $\rangle$ **do**
if $m \notin$ *delivered* **then**
unordered := *unordered* $\cup \{(p, m)\}$;

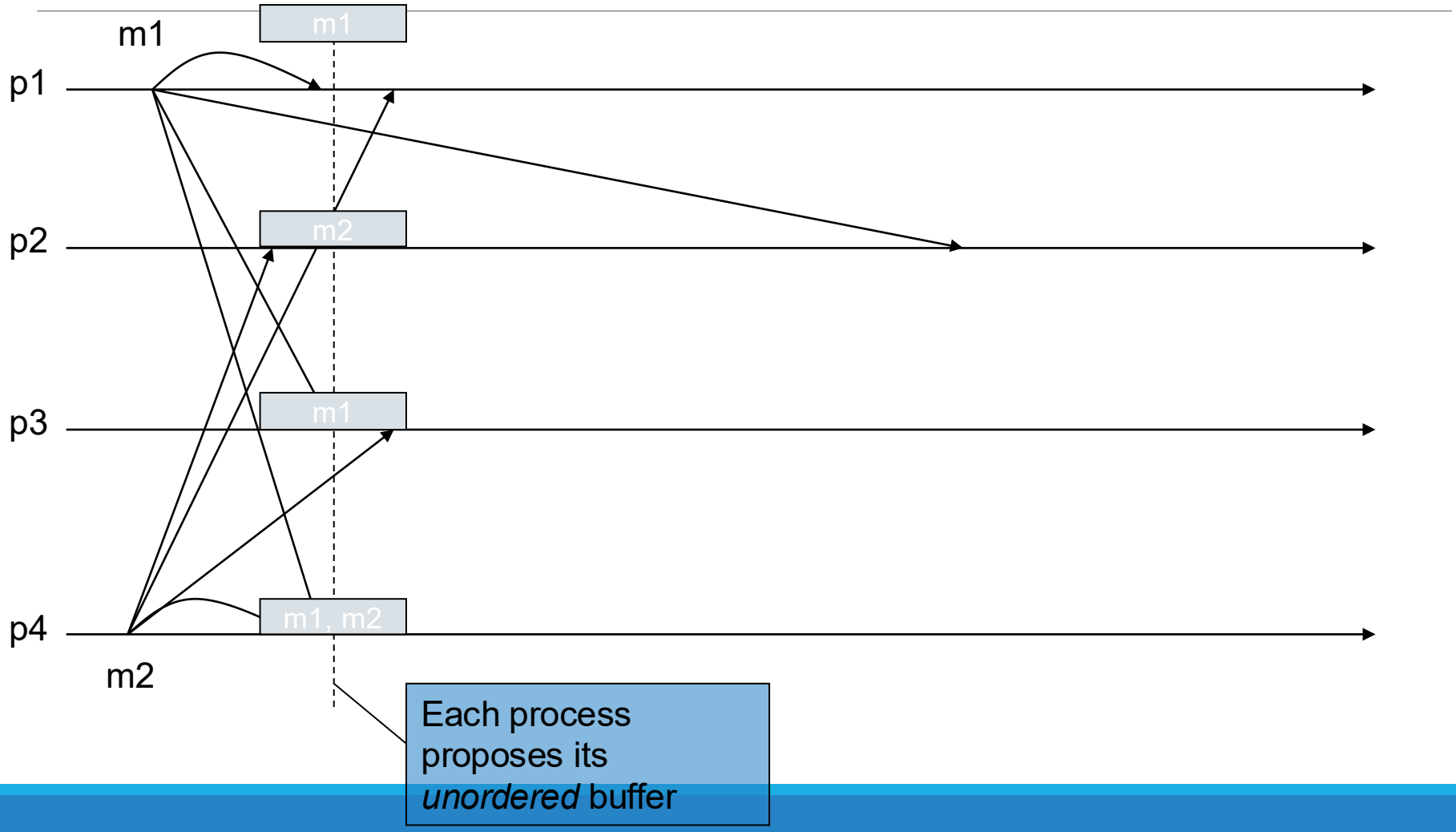
upon *unordered* $\neq \emptyset \wedge$ *wait* = FALSE **do**
wait := TRUE;
Initialize a new instance *c.round* of consensus;
trigger $\langle$ *c.round*, *Propose* | *unordered* $\rangle$;

upon event $\langle$ *c.r*, *Decide* | *decided* $\rangle$ **such that** r = *round* **do**
forall $(s, m) \in$ *sort*(*decided*) **do** // by the order in the resulting sorted list
trigger $\langle$ *tob*, *Deliver* | s, m $\rangle$;
delivered := *delivered* $\cup$ *decided*;
unordered := *unordered* $\setminus$ *decided*;
round := *round* + 1;
wait := FALSE;

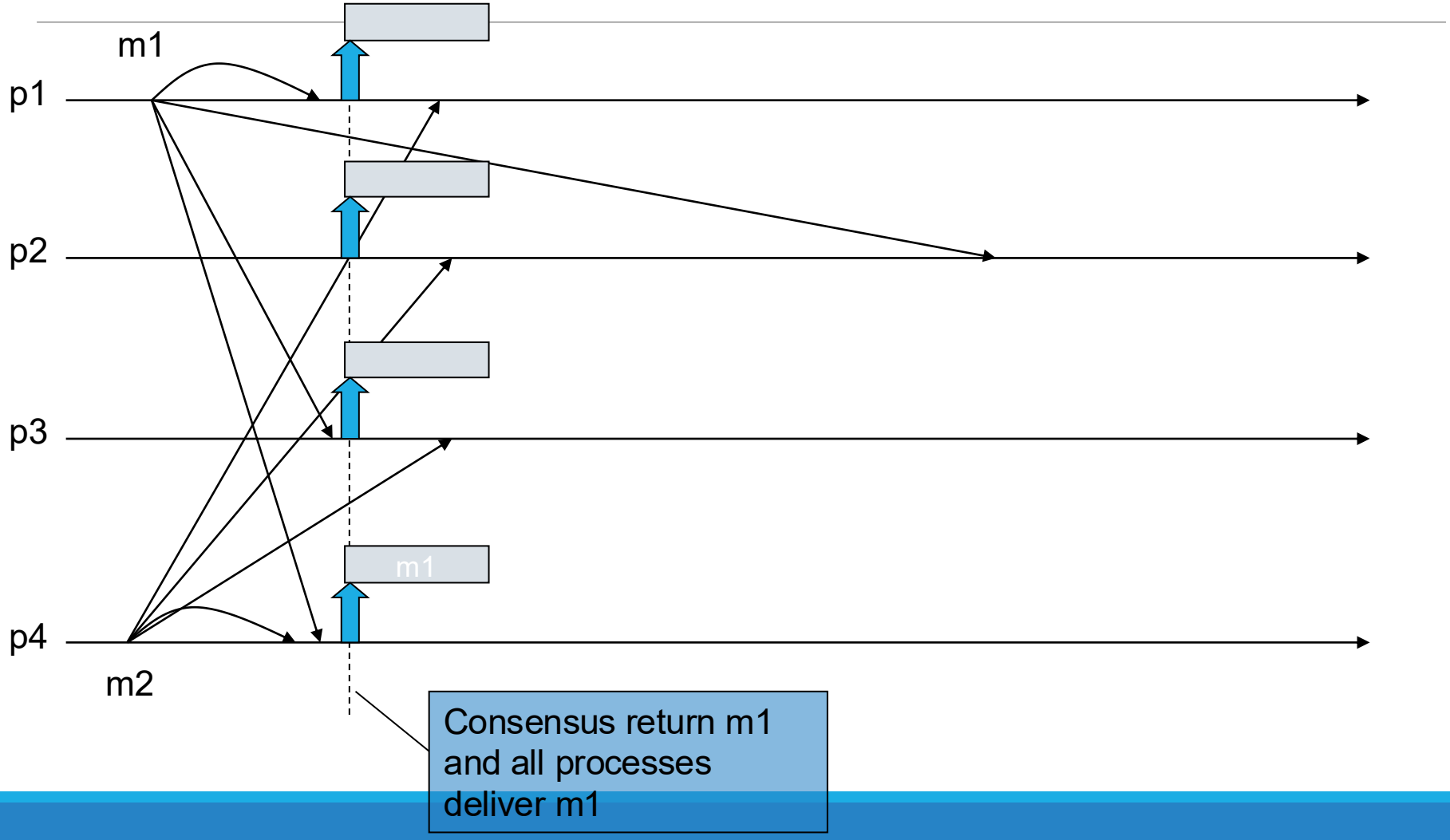
Example



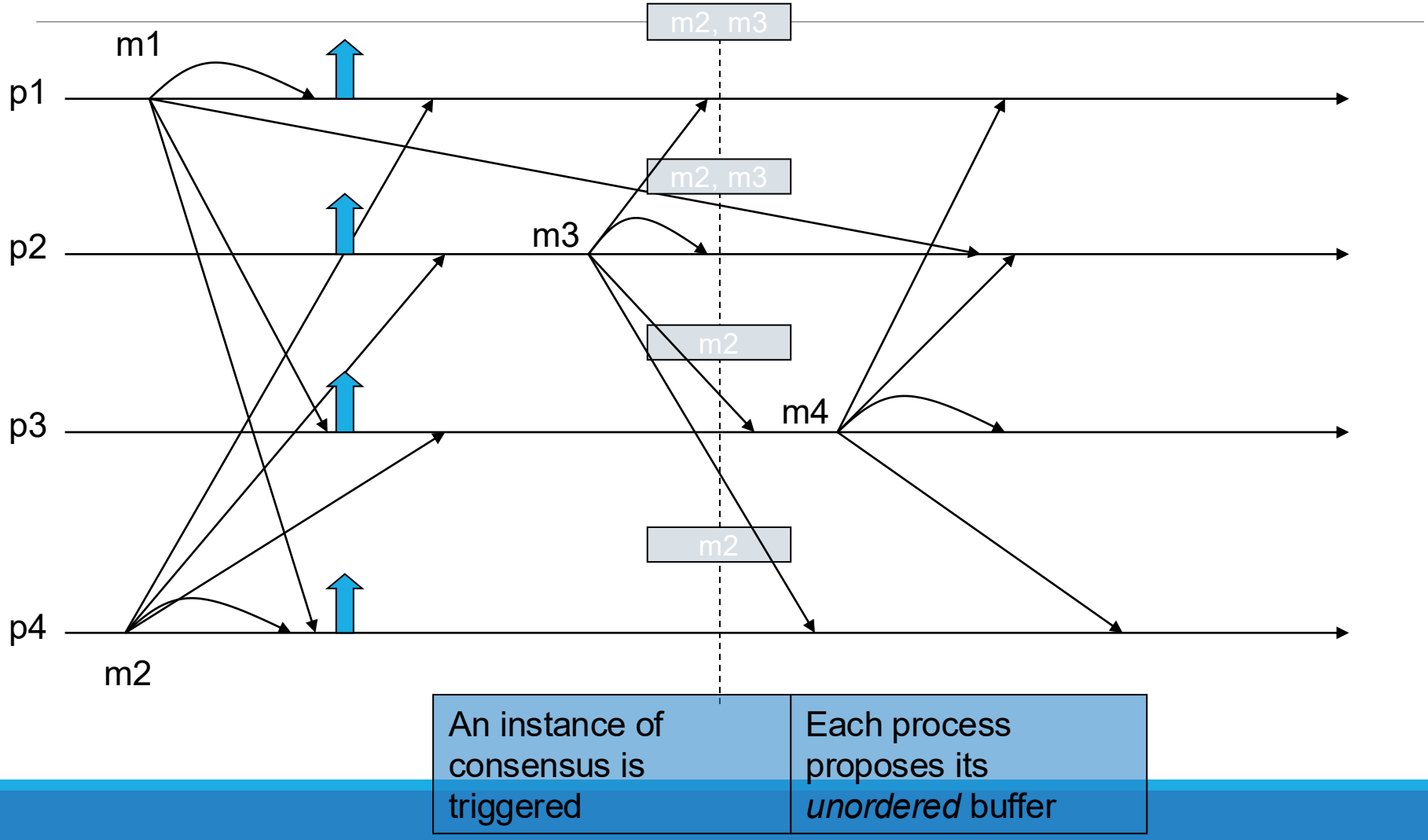
Example


m1
m1
p1
m2
p2
m1
p3
m1, m2
p4
m2
Each process proposes its *unordered* buffer

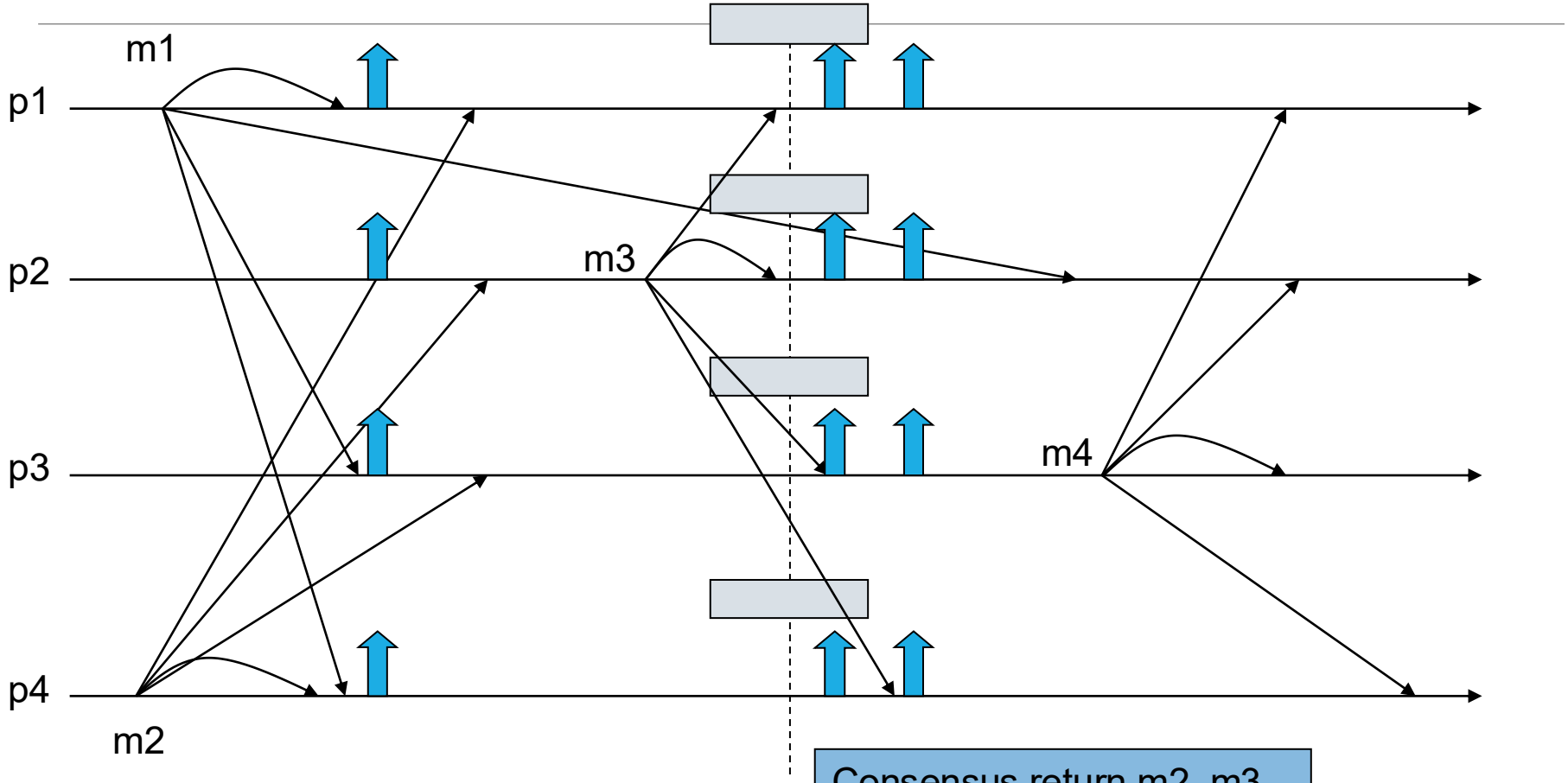
Example


m1
p1
p2
p3
m1
p4
m2
Consensus return m1
and all processes
deliver m1

Example


m2, m3
m1
p1
m2, m3
p2
m3
m2
p3
m4
m2
p4
m2
An instance of consensus is triggered
Each process proposes its *unordered* buffer

Example


m1
p1
p2
m3
p3
m4
p4
m2
Consensus return m2, m3 and all processes deliver m2 and then m3

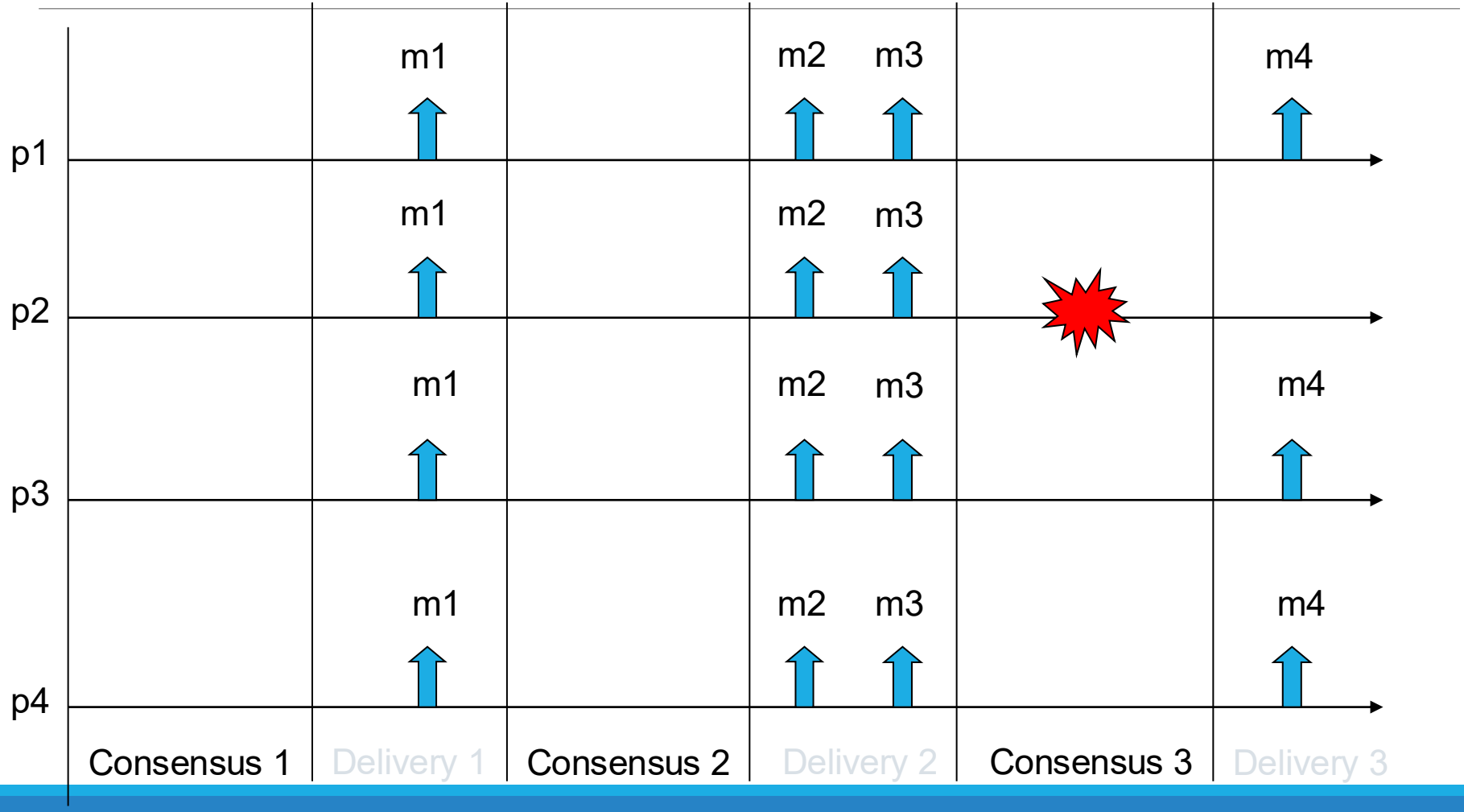
Exercice

Which TO specification is satisfied by this algorithm?

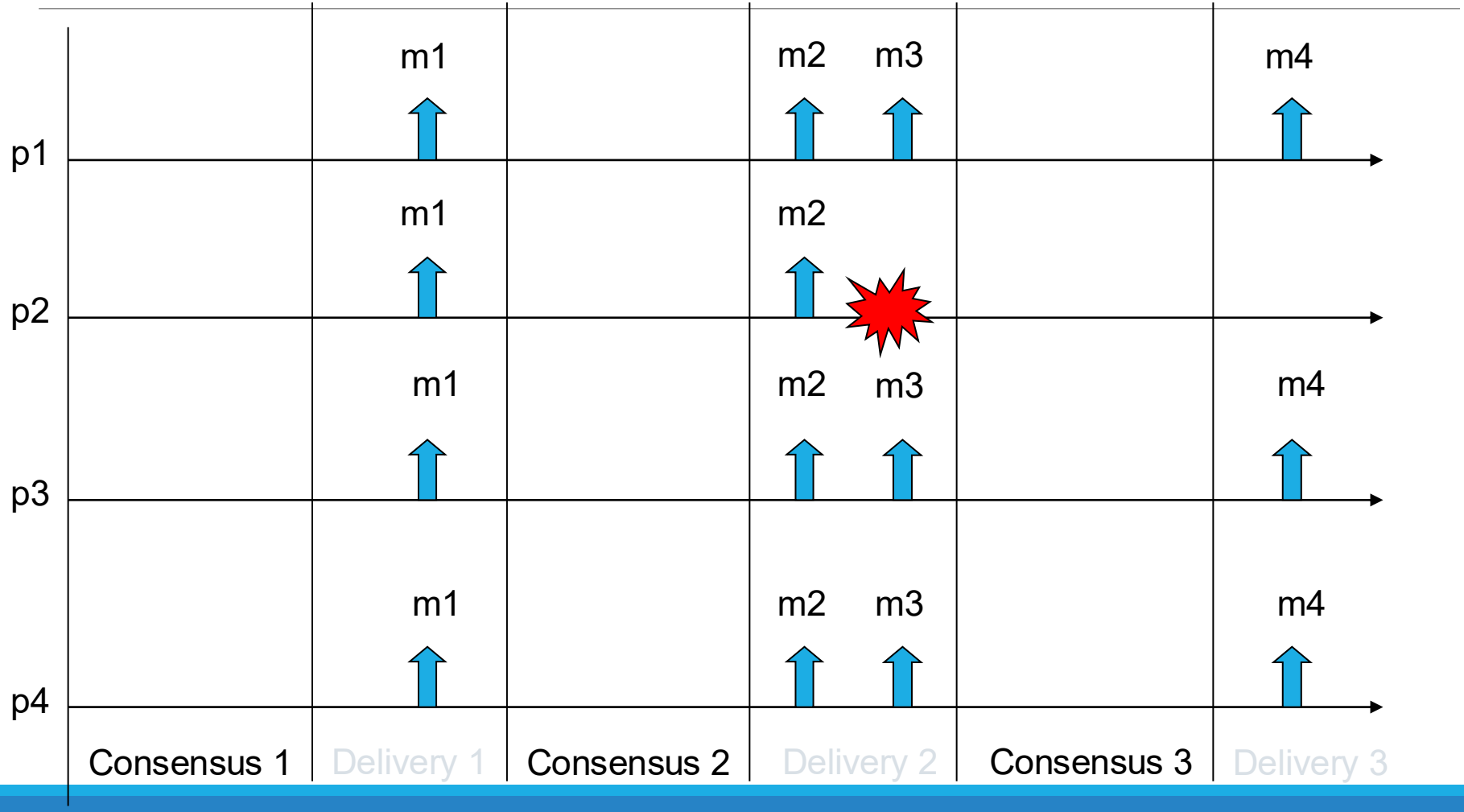
It depends from the assumptions about Reliable Broadcast and Consensus

Consensus / Reliable Broadcast	Uniform	Non Uniform
Uniform		
Non Uniform		

Example 1 (UC and URB)



Example 2 (UC and URB)



Uniform Consensus (UC) and Uniform Reliable Broadcast (URB)

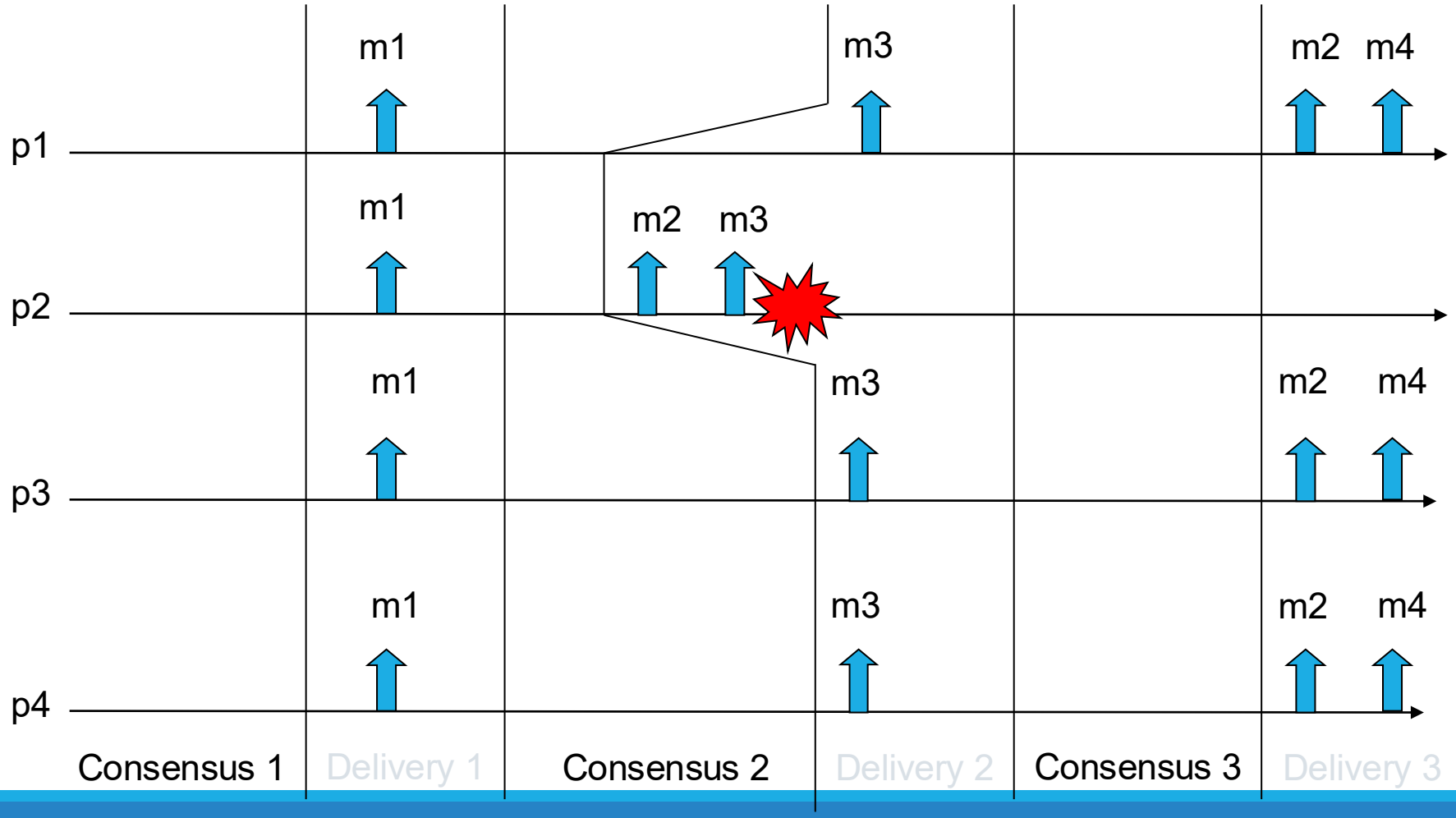
Assuming both Consensus and Reliable Broadcast uniform we have

TO (UA, SUTO)

Proof.

- Due to URB all the processes (even the faults) deliver the same set of messages
- The unordered buffer contains the same set of messages for each process
  - All the processes will deliver the same set of messages (UA)
- Due to UC, all processes (even the faults) decide for the same list of messages
- Messages are sorted by a deterministic rule
  - All processes will deliver the messages in the same order

Example (NUC and URB)



Non Uniform Consensus (NUC) and Uniform Reliable Broadcast (URB)

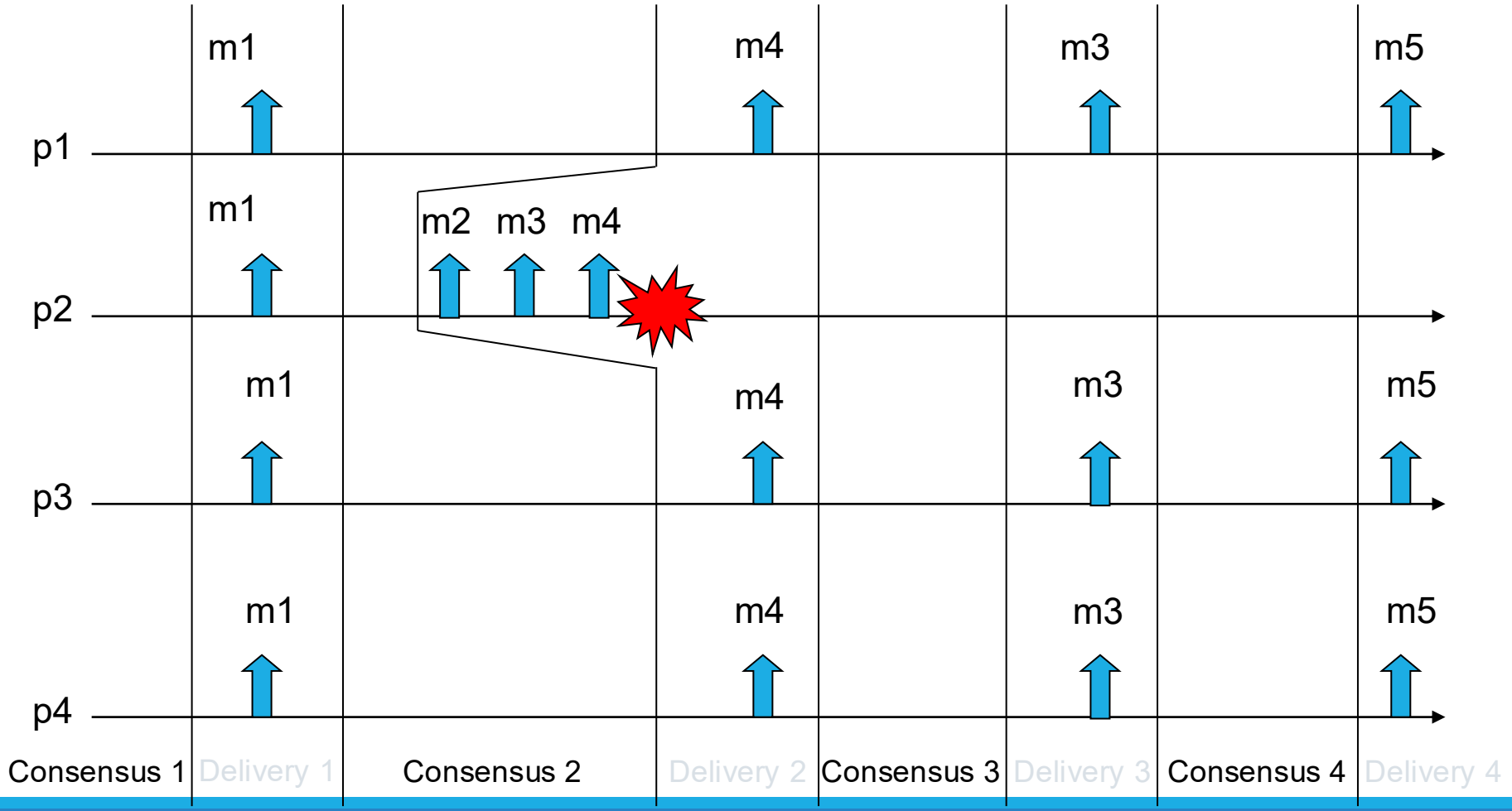
Assuming both Consensus and Reliable Broadcast uniform we have

TO (UA, WNUTO)

Proof.

- Due to URB all the processes (even the faults) deliver the same set of messages
- The unordered buffer contains the same set of messages for each process
  - All the processes will deliver the same set of messages (UA)
- Due to NUC, all correct processes decide for the same list of messages
- Faulty processes can decide differently
  - All correct processes will deliver the messages in the same order
  - Faulty processes will deliver, just before a crash, a different sequence of messages

Example (NUC and NURB)



Non Uniform Consensus (NUC) and Non Uniform Reliable Broadcast (NURB)

Assuming both Consensus and Reliable Broadcast uniform we have

TO (NUA, WNUTO)

Proof.

- Due to NURB correct processes deliver the same set of messages
- Faulty processes can deliver other messages
  - Only correct processes will deliver the same set of messages (NUA)
- Due to NUC, all correct processes decide for the same list of messages
- Faulty processes can decide differently
  - All correct processes will deliver the messages in the same order
  - Faulty processes will deliver, just before a crash, a different sequence of messages

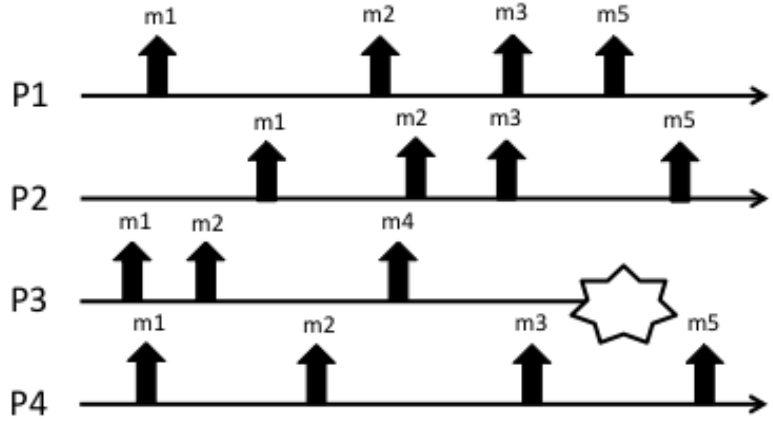
Consensus / Reliable Broadcast	Uniform	Non Uniform
Uniform	UA SUTO	UA WNUTO
Non Uniform		NUA WNUTO

Exercice

Which specification is satisfied assuming UC and NURB?

Exercice

Consider the run depicted in the figure:



1. Which type of total ordering is satisfied by the run? Specify both the agreement and the ordering properties.
2. Modify the run in order to satisfy TO(UA, WUTO) but not TO (UA SUTO)
3. Modify the run in order to satisfy TO(NUA, WNUTO) but not TO(NUA, WUTO)

References

C. Cachin, R. Guerraoui and L. Rodrigues. Introduction to Reliable and Secure Distributed Programming, Springer, 2011

- Chapter 6 – Section 6.1

Stefano Cimmino, Carlo Marchetti, Roberto Baldoni ”*A Guided Tour on Total Order Specifications*” WORDS Fall 2003: 187-194